



WEB SISTEMAK

1. Praktika (2026-02-09): IoT bezeroa

HELBURUA

1. PyCharm IDE-a erabiliz, ThingSpeak-era (<https://thingspeak.com/>) ordenagailuaren uneko %CPU eta %RAM datuak igotzen dituen web bezero bat Python-en programatu behar duzue. Web bezero honek, ondorengo baldintzak bete beharko ditu:
 - a. Bezeroa abiarazten denean, honek burutuko duen lehenengo ekintza bi datu-eremu dituen kanal bat sortzea izango da. Datu-eremuen etiketak **%CPU** eta **%RAM** dira. Bezeroak kanalaren identifikatzailea eta idazteko pasahitza testu fitxategi batean gordeko ditu.
 - b. Bezeroan erregistratutako kanala ThingSpeak-en jada existituz gero, ez da kanal berririk sortuko, hau da, existitzen dena erabiliko da. Bestetik, kanala sortzerakoan erabiltzaileak sortu dezakeen kanal kopuru maximoa gainditu dela detektatzen bada, bezeroak erabiltzaileari gertaera hori jakinaraziko dio, honek arazoa eskuz konpondu dezan. Ondoren bezeroak kanala sorrerarekin jarraituko du.
 - c. Kanala sortu ondoren, bezeroak **15 segundoro** ordenagailuaren uneko **%CPU** eta **%RAM** datuak igoko ditu **psutil** liburutegia erabiliz.
 - d. Datuen igoserari amaiera eman eta bezeroa ixteko, erabiltzaileak **Ctrl+C** sakatuko du. Bezeroak etendura seinale hau kudeatuko du, CSV fitxategi batean kanalean gordetako azken 100 laginak irauliko ditu. Gero bezeroak kanala hustuko du (hau da, kanalean dauden datuak ezabatuko ditu baina kanala bera ezabatu gabe) eta era ordenatuan exekuzioa amaituko du.

Zuen kabuz garatu beharko dituzuen alderdiak:

- JSON kate baten parseoa.
 - Python hiztegi baten prozesamendua.
 - Python-en salbuespenen programazioa.
 - CSV fitxategiak sortzea eta idaztea.
2. HTML web-orri bat sortu ThingSpeak-en gordetako datuak atzitzeko eta Google Charts liburutegia (<https://developers.google.com/chart>) erabiliz horiek grafikoki adierazteko.

Zuen kabuz garatu beharko dituzuen alderdiak:

- JavaScript-en HTTP eskaeren kudeaketa *XMLHttpRequest* objektua erabiliz.
- Datu-taula batean sortzea, HTTP erabiliz deskargatutako datuak irudikatzeko.



ENTREGAGAIK

eGelako zereginera ondorengo entregagaiak igo behar dira, izenaren eta bi abizenen inzialekin identifikatuta (adibidez, Lourdes Pérez Manzano: Practika1_LPM.py eta WebOrria_LPM.html)

1. Web bezeroa implementatzen duen **Python programa** (20. pausua).
2. Datuak grafikoki adierazten dituen **HTML orria**.

Entregagaia identifikatzeko goiburu bat gehituko da hurrengo datuak adieraziz:

- Ikaslearen izen-abizenak
- Irakasgaia eta taldea
- Entregatze data
- Lanaren izena
- Entregagaiaren deskribapen laburra.

PRAKTIKA EGITEKO PAUSUAK ETA ARGIBIDEAK

1. *PyCharm-en proiektu bat sortu*
2. *psutil liburutegia proiektuaren ingurune birtualean instalatu*
3. *Proiektuan Python fitxategi bat sortu*
4. *Python programa baten egitura*
5. *psutil liburutegiaren API-a erabiliz, ordenagailuaren uneko %CPU eta %RAM datuak terminalean 15 segundoro inprimatzen dituen kodea garatu*
6. *Python programa baten exekuzioa*
7. *Ctrl+C etendura kudeatzen duen salbuespena programatu*
8. *ThingSpeak-era lehenengo hurbilketa*
9. *HTTP eskaera baten bidalketaren eta erantzunaren irakurketaren programazioa*
10. *ThingSpeak-en bi datu-eremu dituen kanala sortzeko HTTP eskaera programatu*
11. *Sortu berri den kanalaren datuekin HTTP erantzuna parseatu*
12. *11 eta 12. ataletako kodea batu kanala bat sortzeko eta bere datuak atzitzeko*
13. *12. ataletako programa zalbadu B baldintza betetzeko, hots: kanalaren berrerabilpenaren eta kanalaren kopuru maximoaren salbuespenak kudeatu*
14. *ThingSpeak-era 15 segundoro bi datu igotzen dituen kodea garatu*
15. *13 eta 14. ataletako kodeak batu kanalaren sorrera automatizatzeko B baldintzan adierazitako salbuespenak kontuan hartuz eta 15 segundoro kanala horretara bi datu igotzeko*



16. *ThingSpeak-eko kanal baten azken 100 laginak JSON formatuan deskargatzeko HTTP eskaera programatu*
17. *Hiru datu eremu eta bi datu-erregistro dituen CSV fitxategi bat sortzeko kodea garatu*
18. *ThingSpeak-eko kanal bateko datuak husteko eskaera programatu*
19. *7, 16, 17 eta 18. ataletako kodeak batu, hots: Ctrl+C sakatuz gero, kanal baten azken 100 laginak CSV fitxategi batean irauli eta ondoren kanala hustu*
20. *15 eta 19. ataletako kodeak batu, hots: ThingSpeak-en kanal bat sortu B baldintzan adierazitako salbuespenak kontuan hartuz, kanal horretara %CPU eta %RAM datuak 15. segundoro igo, eta Ctrl+C sakatuz gero azken 100 laginen iraulketa egin CSV fitxategi batean, ThingSpeak-eko kanala hustu eta programaren exekuzioa amaitzen delarik*

OHARRAK:

- 5, 7 eta 10tik 20rako atalak banan-banan, fitxategi banatan lantzea gomendatzen da.
- 20. atalari dagokion fitxategia entregatu beharreko Python programa da.

1. PyCharm-en proiektu bat sortu

Atal hau burutzeko azalpenak **Instalazioa** eskolan eman ziren:

<https://egela.ehu.eus/mod/resource/view.php?id=8581308>

2. psutil liburutegia proiektuaren ingurune birtualean instalatu

Atal hau burutzeko azalpenak **Instalazioa** eskolan eman ziren:

<https://egela.ehu.eus/mod/resource/view.php?id=8581308>

3. Proiektuan Python fitxategi bat sortu

File → New → Python file

4. Python programa baten egitura

Atal hau burutzeko azalpenak **Instalazioa** eskolan eman ziren

<https://egela.ehu.eus/mod/resource/view.php?id=8581308>

5. psutil liburutegiaren API-a erabiliz, ordenagailuaren %CPU eta %RAM datuak terminalean 15 segundoro inprimatzen dituen kodea garatu

Liburutegiaren dokumentazioa:

<https://psutil.readthedocs.io/en/latest/#system-related-functions>

- **%CPU:** %CPU datua lortzeko metodoaren deiak prozesadoreak 15 segunduko tartean duen batazbesteko karga kalkulatzeko du.
- **%RAM:** Erabilitako memoriaren ehunekoa kalkulatzeko, psutil liburutegiaren APIa kontsultatu.

Jarraian, eskatzen den programaren txantilo bat aurkezten da:



```
import psutil
import time

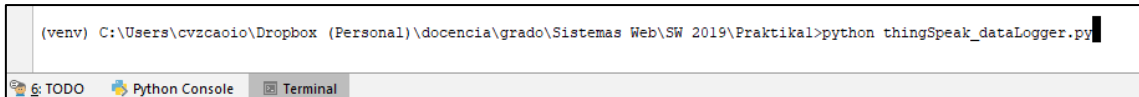
def cpu_ram():
    while True:
        # KODEA: psutil liburutegia erabiliz, %CPU eta %RAM atera
        cpu =
        ram =
        print("CPU: %" + str(cpu) + "%\tRAM: %" + str(ram) + "%")

if __name__ == "__main__":
    cpu_ram()
```

6. Python programa baten exekuzioa

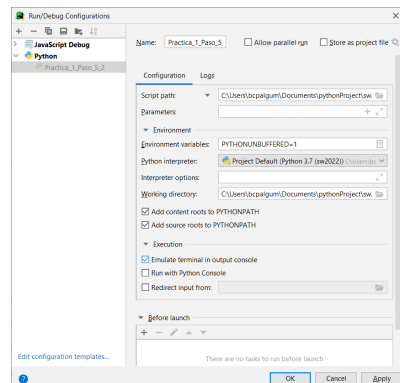
Python programa bat exekutzeko bi bide daude: a) terminala eta b) PyCharm.

a) **Terminala:** PyCharm-en leihoaren azpiko aldean agertzen den *Terminal* botoia sakatu eta bertan Python interpretea deitu programaren izena parametro bezala pasatuz:



Programaren exekuzioa amaitzeko, Ctrl+C sakatu behar da.

b) **PyCharm:** **Run** → **Run...** Kasu honetan, Ctrl+C sakatzeko programaren exekuzioa ez da amaitzen. Zergatik? PyCharm-ek Ctrl+C "kopia" agindu bezela ulertzen duelako. Programa bat **Run leihoan** exekutatzen denean, Ctrl+C amaitze-etendura bezela ulertua izan dadin, PyCharm-en **Run konfigurazioan** "Emulate terminal in output console" gaitu behar da.



7. Ctrl+C etendura kudeatzen duen kodea programatu

Ctrl+C INT seinalea sortzen duen tekla konbinazioa da. Zer da seinale bat? Sistema eragilearen testuinguruan, seinaleak prozesuen arteko komunikazio (IPC) mota bat dira: [https://en.wikipedia.org/wiki/Signal_\(IPC\)](https://en.wikipedia.org/wiki/Signal_(IPC)). INT seinalea terminal batetik prozesu bat etentzeko pentsaturik dago eta, orokorrean, etendura honek prozesuaren amaitze ordenatua dakar.

Jarraian, eskatzen den programa aurkezten da. Programa PyCharm-en idatzi eta exekutatu:



```
import signal
import sys

def handler(sig_num, frame):
    # Gertaera kudeatu
    print('\nSignal handler called with signal ' + str(sig_num))
    print('Check signal number on '
          'https://en.wikipedia.org/wiki/Signal_%28IPC%29#Default_action')

    print('\nExiting gracefully')
    sys.exit(0)

if __name__ == '__main__':
    # SIGINT jasotzen denean, "handler" metodoa exekutatu da
    signal.signal(signal.SIGINT, handler)

    print('Running. Press CTRL-C to exit.')
    while True:
        pass # Ezer ez egin
```

8. ThingSpeak-era lehenengo hurbilketa

eGelako gardenkiak berrikusi: <https://egela.ehu.eus/mod/resource/view.php?id=8581398>.
Burp tresna erabiliz bertan planteatzen diren adibideak burutu.

9. HTTP eskaera baten bidalketaren eta erantzunaren irakurketaren programazioa

Atal honen inguruko azalpenak: <https://egela.ehu.eus/course/view.php?id=94805§ion=2>.

10. ThingSpeak-en bi datu-eremu dituen kanala sortzeko HTTP eskaera programatu

Praktika honetan web bezeroak zenbait HTTP eskaera egingo ditu ThingSpeak-eko REST APIak eskaintzen dituen zerbitzuak eskatzeko.

<https://es.mathworks.com/help/thingspeak/rest-api.html> web orrian ThingSpeak-eko REST API-aren ezaugarriak azaltzen dira, hots:

- HTTP eskaera nola eraiki (zein metodo, zein URI, zeintzu goiburu, zeintzu parametro) ThingSpeak-en prozedura edo zerbitzu jakin bat deitzeko.
- Zein da HTTP erantzunaren edukiaren formatua eta egitura.
- Zein errore gerta daitezke HTTP erantzunaren egoera kodearen arabera.

Adibide gisa, jarraian kanala sortzeko eskaera aurkezten da:

REST API Reference dokumentazioatik abiatuz:

<https://es.mathworks.com/help/thingspeak/rest-api.html>

APIaren web orriari buruzko dokumentazioa ingelesez irakurri: [*“Esta página se ha traducido mediante traducción automática. Haga clic aquí para ver la versión original en inglés.”*](#)

→ Create and Delete → Create Channel

<https://es.mathworks.com/help/thingspeak/createchannel.html>

OHARRA: Kanalak zuen izenaren inzialekin sortuko dituzue. Adibidez, Lourdes Pérez Manzano: lpmKanal



Kanal bat sortzeko HTTP eskaera:

```
POST /channels.json HTTP/1.1
Host: api.thingspeak.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 70

api_key=67PRF2TFLS11MXW3&name=Nire+kanala&field1=%25CPU&field2=%25RAM
```

HTTP eskaera hori sortzeko eta dagokion erantzuna jasotzeko kodea:

```
import requests
import urllib.parse

USER_API_KEY = "67PRF2TFLS11MXW3"

def create_channel():
    metodoa = 'POST'
    uria = "https://api.thingspeak.com/channels.json"
    goiburuak = {'Host': 'api.thingspeak.com',
                 'Content-Type': 'application/x-www-form-urlencoded'}
    edukia = {'api_key': USER_API_KEY,
              'name': 'Nire kanala',
              'field1': "%CPU",
              'field2': "%RAM"}
    edukia_encoded = urllib.parse.urlencode(edukia)
    goiburuak['Content-Length'] = str(len(edukia_encoded))
    print("\n" + uria)
    print(edukia_encoded)
    erantzuna = requests.request(metodoa, uria, headers=goiburuak,
                                data=edukia_encoded, allow_redirects=False)

    kodea = erantzuna.status_code
    deskribapena = erantzuna.reason
    print(str(kodea) + " " + deskribapena)
    edukia = erantzuna.content
    print(edukia)

if __name__ == "__main__":
    print("Creating channel...")
    create_channel()
    print("Channel created. Check on ThingSpeak")
```

OHARRA: Kanala sortzeko, HTTP eskaeraren URI-a parametrizatu daiteke dagokion erantzuna JSON edo XML izan daiten. Erabili JSON formatua.

11. Sortu berri den kanalaren datuekin HTTP erantzuna parseatu

Aurreko ataleko HTTP erantzunak JSON formatua duen datu egitura bat bueltatzen du.

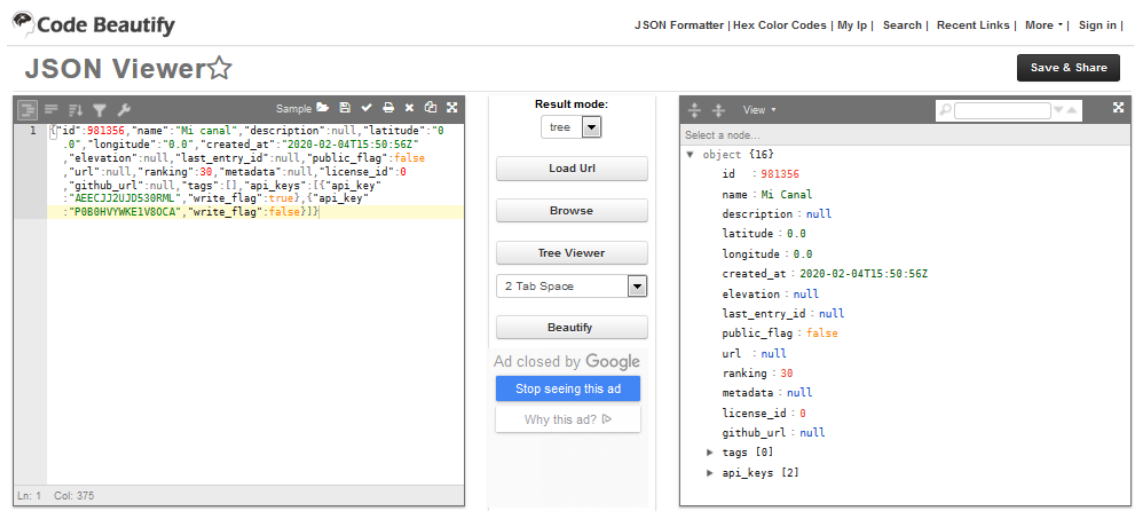
```
{"id":981356,"name":"Nire kanala","description":null,"latitude":"0.0","longitude":"0.0","created_at":"2020-02-04T15:50:56Z","elevation":null,"last_entry_id":null,"public_flag":false,"url":null,"ranking":30,"metadata":null,"license_id":0,"github_url":null,"tags":[],"api_keys":[{"api_key":"AEECJJ2UJD530RML","write_flag":true},{"api_key":"P0B0HVYWKE1V8OCA","write_flag":false}]}
```



JSON datuen egitura antzemateko, bera modu txukunean bistartzeko aukera ematen duen tresna erabiliko dugu: <https://codebeautify.org/jsonviewer>.

Aurreko testu kutxan adierazitako datu egitura kopia eta JSON Viewer tresnan kargatu ezazu. "Tree Viewer" eta "Beautify" aukerak probatu. Hurrengo galderen inguruan hausnartu:

- Zer dago "id" gakoan? Eta "public flag"-en? Zer adierazten du azken gako honek?
- Zer dago "api_keys" gakoan?



JSON formatudun datu egitura bat karaktere kate bat da (string bat, alegia). Egitura horretatik datuak ateratzeko, katea parseatu behar da, hots: JSON sintaxia betetzen ote duen egiaztatu eta datu egitura batean kargatu (adibidez, Python hiztegi batean edo Java HashMap batean):

https://www.w3schools.com/python/python_json.asp

Kanala sortzean itzulitako datu egituratik ondorengoak behar dira:

- Erauzi programatikoki sortu berri den kanalaren ID-a eta idazteko baimenaren gako (WRITE_API_KEY)
- Kargatu bi datu horiek aldagai global banatan, programa exekutatzen denean beren balioak eskuragarri egon daitezkan.
- Gorde datu horiek testu fitxategi batean B baldintza betetzeko.

Python hiztegi bat prozesatzeko azalpenak **Instalazioa** eskolan eman ziren:

<https://egela.ehu.eus/mod/resource/view.php?id=8581308>

OHARRA: Atal hau egiteko, atal honen hasieran aurkeztutako JSON egitura finko batetik abiatzea gomendatzen da, "null", "true" eta "false" baloreak Python-en baliokideei egokitzen direlarik.

12. 11 eta 12. ataletako kodea batu kanal bat sortzeko eta bere datuak atzitzeko

ThingSpeak-en bi eremudun kanal bat sortzeko HTTP eskaera programatu eta dagokion erantzunaren edukia parseatu sortu berri den kanalaren ID-a eta idazteko baimenaren gako (WRITE_API_KEY) lortzeko, hauek aldagai global banatan kargatu eta testu fitxategi batean gorde behar direlarik.



13. 12. ataleko programa zalbadu B baldintza betetzeko

Bezeroan erregistratutako kanala ThingSpeak-en jada existituz gero, ez da kanal berririk sortuko, dagoena erabiliko da, ordea. Horretarako ThingSpeak-eko API-an kanal bat existitzen ote den ziurtatzeko prozedurarik al dagoen egiaztatu behar da, bere identifikatzailetik abiatuz. Hala ez bada, kanal baten existentzia zehazteko logika programatu beharko da, haren identifikatzailetik abiatuta, beste prozeduraren bat erabiliz.

Bestetik, kanala sortzerakoan erabiltzaileak sortu dezakeen kanal kopuru maximoa gainditu dela detektatzen bada, bezeroak erabiltzaileari gertaera hori jakinaraziko dio, honek arazoa eskuz konpondu dezan. ThingSpeak-ek erabiltzaile baten kontuari esleitutako kanal kopuru maximoa alda daitekeenez, erabiltzaile batek kopuru maximoa gainditu ote duen egiaztatzeke, kanal bat sortzerakoan ThingSpeak-ek bueltatzen duen HTTP erantzunaren egoera kodea eta deskribapena aztertzea gomendatzen da.

14. ThingSpeak-era 15 segundoro bi datu igotzen dituen kodea garatu

Zehaztu helburu honetarako zein den ThingSpeak API-ak eskaintzen duen prozedura eta bere parametrizazioa.

OHARRAK:

- Ez da 8. ataleko programa hedatu behar. Hau da, %CPU eta %RAM balioak igo beharrean, kodean eskuz sartutako edozein bi balio igo.
- Ez da 14. ataleko programa abiapuntutzat erabili behar. Hau da, ez beharrezkoa ThingSpeak-eko API-a erabiliz kanal bat sortzea datuak igo baino lehen. Hori egin beharrean, eskuz sartuko da aurretik sortutako kanal baten idazteko baimenaren gakoa (WRITE_API_KEY)

15. 13 eta 14. ataletako kodeak batu

Kanalaren sorrera automatizatu B baldintzako salbuespenak kontuan hartuz eta 15 segundoro kanal horretara bi datu igo.

16. ThingSpeak-eko kanal baten azken 100 laginak JSON formatuan deskargatzeko HTTP eskaera programatu

Zehaztu hau egiteko zein den ThingSpeak-eko API-ak eskaintzen duen prozedura eta bere parametrizazioa. Erantzunak 400 (Bad Request) egoera kodea itzuliz gero, zeri egotz dakioko? Zeintzu bi modu daude hori konpontzeko?

17. Hiru datu eremu eta bi datu-erregistro dituen CSV fitxategi bat sortzeko kodea garatu

Python-en `csv` liburutegia erabili. Atal hau datu finkoekin egin. Adibidez :

```
1 timestamp,cpu,ram
2 2023-01-27T13:07:48Z,24.0,56.7
3 2023-01-27T13:08:03Z,21.7,55.3
4
```




18. ThingSpeak-eko kanal bateko datuak husteko eskaera programatu

Zehaztu hau egiteko zein den ThingSpeak-eko API-ak eskaintzen duen prozedura eta bere parametrizazioa.

19. 7, 16, 17 eta 18. ataletako kodeak batu

Ctrl+C sakatzean, ThingSpeak-i kanal bateko azken 100 laginak JSON formatuan itzultzeko HTTP eskaera egin, JSON datuak Python-en hiztegi batean irauli, hiztegi hau CSV fitxategi batean irauli eta kanala datuz hustu.

OHARRA: 15. ataletako programa terminal batean exekutatu kanala datuekin elikatzeko.

20. 15 eta 19. ataletako kodeak batu

Programa bat sortu ondorengo ezaugarriekin:

- Hasiera: ThingSpeak-en kanal bat sortu B baldintzaren salbuespenak kontuan hartuz.
- Begizta nagusia: kanal horretara %CPU eta %RAM datuak 15. segundoro igo.
- Amaiera: Ctrl+C sakatuz gero, azken 100 laginen iraulketa egin CSV fitxategi batean, ThingSpeak-eko kanala hustu eta programaren exekuzioa modu ordenatuan amaitu.

PRAKTIKAREN BIGARREN HELBURUAREN JARRAIBIDEAK ETA ATALAK

- 1. LineChart.html adibidea ireki eta aztertu*
- 2. HTML batetik ThingSpeak-i egindako HTTP eskaeraren adibidea aztertu*
- 3. ThingSpeak-etik deskargatutako datuak grafikoki adierazten dituen HTML orria sortu*

1. LineChart.html adibidea ireki eta aztertu

HTML orri honetan JavaScript-en programatutako Google Charts liburutegia erabiltzen da. Kodea eGelan eskuragarri dago: <https://egela.ehu.eus/mod/resource/view.php?id=8581399>.

2. HTML batetik ThingSpeak-i egindako HTTP eskaeraren adibidea aztertu

HTML orri batetik HTTP eskaera bat egiteko JavaScript kodea erabiltzen da, zehazki XMLHttpRequest objektua. Honi buruzko dokumentazioa ondorengo esteketan eskura daiteke:

https://www.w3schools.com/xml/ajax_xmlhttprequest_create.asp
https://www.w3schools.com/xml/ajax_xmlhttprequest_response.asp
<https://developer.mozilla.org/es/docs/Web/API/XMLHttpRequest>

OHARRA: HTTP eskaera, modu asinkronoan egin behar da, hau da, HTTP eskaera egin ondoren, JavaScript kodea ezin daiteke erantzunaren zain blokeatuta gelditu.



Jarraian ematen den adibidetik abiatuta, galdera hauei buruz hausnartu:

- Noiz exekutatzen da *XMLHttpRequest* objektuaren *onreadystatechange* atributuari lotutako funtzioa, HTTP eskaera egin aurretik edo ondoren?
- Zer adierazten du *readyState* atributuak?
- Non adierazten da HTTP eskaera modu asinkronoan egin behar dela?

<https://egela.ehu.eus/mod/resource/view.php?id=8581400>

```
<html>
<head>
  <script type="text/javascript">
    function feedsData() {
      var xhttp = new XMLHttpRequest();
      var uri = "https://api.thingspeak.com/channels/1897127/feeds.json";

      xhttp.onreadystatechange = function() {
        if (this.readyState == 4 && this.status == 200) {
          // ver los datos en crudo (en formato JSON) en la página
          document.write(xhttp.responseText);

          // ver los datos parseados en la consola
          var responseData = JSON.parse(xhttp.responseText);
          console.log(responseData);
        }
      };

      // inicializar la petición HTTP
      xhttp.open("GET", uri, true);
      // enviar la petición HTTP
      xhttp.send();
    };
  </script>
</head>
<body onload="feedsData()">
</body>
</html>
```

3. ThingSpeak-etik deskargatutako datuak irudikatzen dituen HTML orria sortu

Aurreko bi ataletik abiatuz, sortu kanaleko datuak irudikatuko dituen HTML orria.

Datuak “on-line”, ThingSpeak-i bidalitako HTTP eskaera baten bidez, lortu behar diranez, datu-taularen erregistroak (errenkadak) ezin izango dira *hardcoded* sartu 1. atalean bezala, baizik dinamikoki sortu behar den *DataTable* objektu baten bitartez.

<https://developers.google.com/chart/interactive/docs/reference#DataTable>.

Gainera, hurrengo bistaratzeko aukerak erabiliko dira:

```
// OPCIONES DE VISUALIZACIÓN: EJE DE ORDENADAS PARA CADA VARIABLE
var options = {
  title: 'Computer performance', legend: {position: 'bottom'},
  curveType: 'function', colors: ['red', 'blue'],
  series: {0: {targetAxisIndex: 0}, 1: {targetAxisIndex: 1}},
  vAxes: {0: {title: '%CPU'}, 1: {title: '%RAM'}}
};
```